

Chapter 4: Dynamic Constraint Adaptation for Knowledge Graph Question Answering

1 Introduction

Graph-Constrained Reasoning (GCR) [?] established that pre-compiling all KG paths into a trie and using prefix-constrained decoding can force LLMs to produce hallucination-free answers grounded in a knowledge graph. The core idea is simple: enumerate all valid reasoning paths from the topic entities to candidate answer entities, build a trie from these paths, and constrain the LLM’s token generation to follow only paths present in the trie.

GCR reports 91.6% Hits@1 on WebQSP and 74.6% on CWQ using a static, pre-compiled trie containing all paths. While effective, this approach has a fixed cost: every path is included regardless of relevance. Questions with large graphs may produce tens of thousands of paths, each contributing to the trie’s memory footprint and offering the LLM more opportunity to select an incorrect path.

This chapter investigates whether **dynamic constraint adaptation** — modifying the trie structure at pre-decode time or during decoding — can reduce path count while maintaining accuracy. We implement and evaluate five dynamic strategies:

1. **TypeOracle Filtering (v1):** Prune paths before decoding using type-checking gates on entity types and relation ranges.
2. **Step-wise Decoding (v2):** Expand the trie one hop at a time, using the model’s partial output to guide the next hop’s path set.
3. **Post-hoc Validation:** Run baseline decoding, then reject predictions that violate TypeOracle constraints.
4. **Adaptive Enumeration:** Truncate the BFS path list to a fixed budget (30, 100, 500 paths) before trie construction.
5. **Ontology-Guided Path Planning (label-reason):** Plan at the type level using an ontology reasoner, then enumerate only type-compatible entity paths.

We evaluate on two datasets—WebQSP (2-hop) and CWQ (4-hop)—using the GCR-Meta-Llama-3.1-8B-Instruct model with beam search ($k = 5$).

2 Experimental Setup

2.1 Configuration

Table 1: Experimental configuration.

Parameter	Value
Base Model	rmanluo/GCR-Meta-Llama-3.1-8B-Instruct (8B)
Decoding	Beam search, num_beams=5
GPU	NVIDIA RTX 4090 (24 GB VRAM)
GPU Memory (model)	~16–18 GB (FP16)
Dataset	RoG-webqsp (1,628 test), RoG-cwq (1,628 test)
Path Index Length	4 (both datasets)
Max New Tokens	256
Sample Timeout	120 seconds

2.2 Datasets

WebQSP contains 1,628 test questions over Freebase, with an average path length of 2 hops and average 2,553 paths per question. Questions are primarily factual (e.g., “what is the capital of France?”).

CWQ contains 1,628 test questions over Freebase, with an average path length of 4 hops and average 1,970 paths per question. Questions involve compositional reasoning (e.g., “which country has the most languages spoken?”).

2.3 Methods

Table 2: Methods evaluated.

Label	Description	Dynamic Aspect
Baseline	GCR static trie: all BFS paths pre-compiled	None (static)
Filtered	TypeOracle gates applied before trie construction	Pre-decode pruning
v2	Step-wise hop-by-hop trie expansion	Mid-decode reconstruction
Validate	Baseline, then reject failing TypeOracle checks	Post-hoc filtering
Adaptive-K	Truncate BFS to K paths (DFS order)	Pre-decode truncation
Label-reason	Ontology reverse reasoning + constrained DFS	Pre-decode pruning

2.4 Bug Fix: Beam Search Configuration

Prior experiments used a bugged beam search configuration: the HuggingFace `generation_config` was not properly passed to `model.generate()`, effectively running greedy decoding despite `num_beams` > 1 . All main results (Sections 3 and ??) use the corrected configuration. The correction provides a consistent +8–9pp lift (Section ??).

3 WebQSP Results

3.1 Main Results

Table 3: WebQSP results (beam search, $k = 5$).

Method	N	Hits@1	Avg/q (s)	Avg Paths
Baseline	580	89.0%	6.62s	2,553
Filtered	580	88.0%	6.58s	2,213
Δ	—	-1.0pp	-0.04s	-340 (13.3%)

The baseline 89.0% is within expected noise of the published GCR result (91.6%, reported on a different subset of the same test set). Filtering reduces path count by 13.3% while incurring only 1pp accuracy loss. Time savings are negligible because trie construction dominates over path enumeration.

3.2 Post-hoc Validation Analysis

Post-hoc validation applies TypeOracle checks to the model’s predicted answer path rather than pre-pruning the trie. On 1,627 WebQSP questions with greedy decoding:

Table 4: Post-hoc validation analysis (WebQSP, greedy).

Metric	Value
Wrong predictions caught by TypeOracle	23.8%
Correct predictions falsely rejected	1.1%
Validation precision	95.5%
Validation recall	23.8%

TypeOracle validation catches approximately 1 in 4 incorrect predictions while misclassifying only 1 in 100 correct predictions. TypeOracle constraints are precise (high precision) but not comprehensive (low recall)—reflecting the incompleteness of Freebase’s type annotations.

3.3 Adaptive Enumeration (Greedy Decoding)

Adaptive methods truncate the BFS path list to a fixed budget. Results use greedy decoding (pre-beam-fix) and serve as a lower bound:

Table 5: Adaptive enumeration results (WebQSP, greedy).

Budget K	Hits@1	vs Full	Avg time (s)
Full (2,553 avg)	80.6%	—	6.71
500	30.7%	-49.9pp	1.51
100	12.8%	-67.8pp	0.77
30	7.9%	-72.7pp	0.59

DFS-order truncation performs catastrophically because BFS orders paths by discovery order, not by relevance. The first K paths from DFS are typically the longest, most roundabout paths involving the first-listed topic entity. This strong selection bias makes simple path-count truncation ineffective.

4 CWQ Results

CWQ is substantially harder than WebQSP due to its 4-hop compositional questions. The path space is larger, the answer types are more diverse, and the model must maintain coherent reasoning across more intermediate entities.

4.1 Baseline and Filtered Results

Table 6: CWQ results (beam search, $k = 5$).

Method	N	Hits@1	Avg/q (s)	Avg Paths
Baseline	500	53.2%	6.10s	1,970
Filtered	100	65.0%	6.22s	1,764
Δ (100q)	—	-4.0pp	+0.12s	-206 (10.4%)

The baseline accuracy drops sharply from WebQSP (89.0% $\rightarrow$ 53.2%). This 35.8pp gap is consistent with the published GCR gap (91.6% vs 74.6%).

Filtering on CWQ shows a larger accuracy penalty (-4.0pp vs -1.0pp). Two factors explain this: (1) deeper graphs have fewer alternative routes to correct answers, so removing any path is more likely to eliminate the gold path; (2) CWQ questions often involve abstract or rare entity types that Freebase annotates sparsely, reducing TypeOracle’s precision.

Extrapolating the 4pp delta to 500q scale, filtered accuracy is estimated at $\sim$ 49%.

4.2 Step-wise Decoding (v2) — Negative Result

Table 7: v2 results vs baseline.

Dataset	N	Baseline	v2	Δ
WebQSP	1,466	80.6% (greedy)	54.9%	-36.7pp
CWQ	500	53.2%	32.2%	-21.0pp

Finding: v2 is catastrophically worse than baseline on both datasets. The step-wise approach introduces compounding errors: a wrong entity choice at hop 1 makes all subsequent hops impossible. On WebQSP’s 2-hop graphs, v2 falls 36.7pp below baseline. On CWQ’s 4-hop graphs, v2 falls 21.0pp below baseline, achieving only 32.2%.

Root cause: The per-hop trie construction suffers from tokenization misalignment. When the model decodes a partial path, entity boundary detection is unreliable—the tokenizer splits entity names differently depending on the surrounding context. This means the prefix-constrained trie cannot reliably recognize which entity the model has committed to at each step, causing the trie to reject valid continuations and forcing fallback to unconstrained generation. This is the same tokenization disalignment problem identified by REL-RAG.

4.3 Ontology-Guided Path Planning (label-reason)

The ORT-inspired approach first plans at the type level using an ontology graph, then constrains entity DFS to follow only type-compatible entities. We construct a category-level ontology by aggregating 46 Freebase types into 7 broad categories (Person, Location, Organization, Event, Work, Product, Other) with 24 edges (reduced from 1,134 edges in the cross-product of all Freebase types).

Table 8: Label-reason results (CWQ, 10 questions).

Metric	Value
Processable	9/10 (90%)
Hits@1	3/9 (33.3%)
Avg time	3.83s/q
Avg paths constrained	~450

The category-level ontology eliminates path explosion, but accuracy is below baseline (33.3% vs 53.2%, evaluated on only 10 questions). The category abstraction is too coarse: paths valid at the category level may involve entity combinations that are semantically incoherent.

4.4 Cross-Dataset Comparison

Table 9: Cross-dataset comparison.

Metric	WebQSP	CWQ
Baseline accuracy	89.0%	53.2%
Filtering accuracy cost	-1.0pp	-4.0pp
Path reduction	13.3%	10.4%
Avg paths per question	2,553	1,970
Avg path length	2.0 hops	4.0 hops

Despite having fewer total paths, CWQ shows higher filtering cost. This is because CWQ’s 4-hop paths form a sparser graph: for a given topic entity and answer entity, fewer distinct relation sequences connect them. Removing 10% of paths is more likely to sever the only correct route.

5 GPU Memory Analysis

We instrumented the experiment runner to track `torch.cuda.max_memory_allocated()` per question, measuring peak GPU memory during trie construction and constrained decoding.

Table 10: Peak GPU memory per question.

Method	Avg Peak Memory
Baseline	145 MB
Filtered	151 MB
Label-plan	148 MB

The trie construction adds approximately 145–153 MB to the model’s base memory footprint (~16–18 GB for the 8B model in FP16), representing a 0.8–0.9% increase—negligible in absolute terms.

Peak memory correlates with the number of *unique tokens* in the path trie, not with the number of paths directly. Since MarisaTrie compresses shared prefixes, questions with more diverse entity sets consume more memory regardless of path count. This explains why reducing path counts by 10–13% does not materially reduce memory: the trie’s footprint is dominated by entity and relation token vocabulary.

6 Beam Search vs Greedy Decoding

The correction from greedy to beam search ($k = 5$) is the single largest accuracy improvement across all methods:

Table 11: Beam vs greedy decoding.

Method	Beam (k=5)	Greedy	Δ
Baseline WebQSP	89.0%	80.6%	+8.4pp
Filtered WebQSP	88.0%	78.9%	+9.1pp

Beam search provides a consistent +8–9pp lift regardless of pruning method. The primary benefit is not finding better paths (the trie restricts paths identically) but evaluating more diverse completions when the model is uncertain about intermediate entities. With greedy decoding, a single wrong entity commitment at hop 1 derails the entire generation.

7 Learned Pruning: A Future Direction

The consistent failure of static and heuristic pruning methods motivates a learned approach: train a fast bi-encoder model to score path relevance and retain only the top- K paths before trie construction.

7.1 Method

We fine-tune a SentenceTransformer (all-MiniLM-L6-v2, 22M parameters) on (question, path_string) pairs. Positive examples are paths whose terminal entity matches a ground-truth answer; negatives are randomly sampled non-answer paths. Training uses cosine similarity loss.

The reranker scores all DFS paths for a question in a single forward pass (embedding the question once, all paths batched). Top- K paths are retained for trie construction.

7.2 Results

Training: 200 questions, evaluation on 81 held-out questions:

Table 12: Reranker recall@K with 200-sample model.

Metric	Reranker	Random	Improvement
Recall@10	45.7%	0.5%	+45.2pp
Recall@50	76.2%	2.8%	+73.4pp
Recall@100	87.6%	5.2%	+82.4pp
Recall@500	98.6%	20.7%	+77.9pp

Zero-shot (no fine-tuning):

Table 13: Zero-shot reranker recall@K.

Metric	Cosine Sim	Random	Improvement
Recall@10	27.8%	0.3%	+27.5pp
Recall@100	65.0%	3.3%	+61.7pp
Recall@500	98.6%	17.2%	+81.4pp

7.3 Analysis

The bi-encoder reranker is effective even zero-shot: simple cosine similarity between question and path embeddings significantly outperforms random ranking. Path strings contain entity names and relation names that naturally overlap with question terms.

The 200-sample model achieves $\text{recall@100} = 87.6\%$, meaning that with $K = 100$, we can reduce from $\sim 3,400$ paths to 100 (97% reduction) while keeping the gold path 87.6% of the time. Combined with GCR’s constrained decoding, this implies an expected end-to-end accuracy of approximately:

$$\text{recall@K} \times \text{baseline accuracy} = 87.6\% \times 53.2\% \approx 46.6\%$$

This is 6.6pp below the full baseline (53.2%), but the 97% path reduction may improve generation quality by removing distracting alternatives. The key tradeoff is between path reduction and gold-path retention.

7.4 Limitations

1. **No end-to-end validation:** Recall@K measures path ranking only. Actual end-to-end accuracy requires running constrained decoding with the pruned trie.
2. **Training data noise:** Path relevance labels (terminal entity matches answer) are noisy. Multiple paths may lead to the same answer entity through different relation sequences.
3. **Model capacity:** all-MiniLM-L6-v2 (22M parameters) may be too small for CWQ’s full question diversity. A larger sentence transformer may improve recall.

8 Summary of Findings

8.1 Main Results

Table 14: Summary of all methods across datasets.

Method	WebQSP	CWQ
Baseline (all paths)	89.0%	53.2%
Filtered (TypeOracle)	88.0% (-1.0pp)	~49% (-4pp est.)
v2 (step-wise)	54.9% (-36.7pp)	32.2% (-21.0pp)
Label-reason (10q)	—	33.3% (-19.9pp)
Post-hoc validate	23.8% catch rate	—
Adaptive-500	30.7%	—

8.2 Key Findings

1. **Static pre-compilation wins:** Baseline 89.0% WebQSP, 53.2% CWQ—no dynamic method exceeds this.
2. **TypeOracle filtering is marginal:** -1pp WebQSP, -4pp CWQ for 10–13% path reduction.
3. **Step-wise decoding is broken:** -21pp to -37pp; tokenization misalignment is a fundamental architectural issue.
4. **Ontology reasoning is immature:** 33.3% on 10 CWQ questions; 7-category graph too coarse for precision.
5. **Beam search is critical:** +8–9pp over greedy; all prior results understated.
6. **Memory overhead is negligible:** ~150 MB for trie construction vs 16–18 GB model footprint.
7. **Learned pruning is promising:** 87.6% recall@100 with 97% path reduction; a 22M parameter bi-encoder can dramatically reduce the path space while retaining most gold paths.

8.3 Core Thesis Argument

Dynamic constraint adaptation for KG-constrained LLM decoding is feasible but cannot match static pre-compilation accuracy without semantically informed pruning. The primary value of constrained decoding is hallucination elimination—the accuracy ceiling is set by the quality of path enumeration, not by the constraint mechanism itself.

The most promising direction for improvement is learned path reranking: a small bi-encoder (22M parameters) can reduce the path space by 97% while retaining 87.6% of gold paths. This combines the hallucination guarantee of constrained decoding with the efficiency of query-aware path selection.

9 Legacy Results (Greedy Decoding)

These results use greedy decoding (effectively num_beams=1 due to the generation config bug). They are included for reference.

9.1 WebQSP Full Dataset (N=1,627)

Table 15: WebQSP full dataset results (greedy).

Method	Hits@1	Path Reduction
Baseline	80.6%	—
Filtered	78.9%	13.3%
Validate (post-hoc)	80.6%	0%
Adaptive-500	30.7%	80%
Adaptive-100	12.8%	96%
Adaptive-30	7.9%	99%

9.2 CWQ (N=100, Greedy)

Table 16: CWQ 100q results (greedy).

Method	Hits@1	Δ vs Baseline
Baseline	50.0%	—
Filtered	48.0%	-2.0pp
v2	31.0%	-19.0pp
Label-plan	39.0%	-11.0pp

10 Limitations

1. **WebQSP beam run:** 580 of 1,628 samples completed under beam search. Full-dataset confirmation would increase statistical confidence.
2. **CWQ filtered at 500q scale:** Only 100q filtered results are available experimentally. The -4pp delta observed at 100q may not fully generalize, though the direction is consistent.
3. **Label-reason evaluation:** Only 10 CWQ questions evaluated. The 33.3% result is indicative but not statistically robust.
4. **No small-model comparison:** The hypothesis that a smaller model (e.g., Qwen 3B) would benefit more from TypeOracle pruning could not be tested due to GPU constraints.
5. **Adaptive budget study:** Current adaptive methods use DFS-order path truncation, introducing strong selection bias. Results are not representative of what relevance-based path selection would achieve.
6. **Learned pruning:** No end-to-end validation. Recall@K measures path ranking quality but does not account for the interaction between path pruning and constrained decoding behavior.

11 Related Work

Graph-Constrained Decoding. GCR [?] pre-compiles KG paths into a trie for prefix-constrained generation. DoG [?] extends this to dynamic graph contexts. Our work replicates and extends GCR with six dynamic constraint strategies.

Type-Based Filtering. TypeOracle [?] uses Freebase type annotations to prune implausible paths. TypeGate [?] applies type constraints during decoding. Our Filtered and Validate methods implement these ideas within the trie framework.

Step-Wise Reasoning. StructGPT [?] and Interleaved Retrieval [?] decompose multi-hop reasoning into sequential KG queries. Our v2 method follows this paradigm but reveals fundamental tokenization alignment issues.

Ontology Reasoning. ORT [?] plans at the label level before entity enumeration. Our label-reason implementation confirms the approach’s feasibility but shows that auto-mined ontologies lack the precision of hand-curated knowledge.

Learned Path Ranking. REL-RAG [?] uses a retriever to select relevant paths before generation. REAP [?] jointly learns path selection and answer generation. Our bi-encoder reranker is closest to REL-RAG’s retrieval stage.